# Power BI Module end project- Fleet Performance and Delivery Efficiency Dashboard

**Dataset:** Power_BI_module_end_assignment_dataset-logistics_project_dataset.xlsx

**Sheets:** Trip_Data (50 real trips, ~950 blank rows below them) and Vehicle_Master (7 vehicles)

## STEP 1 — Load the Data

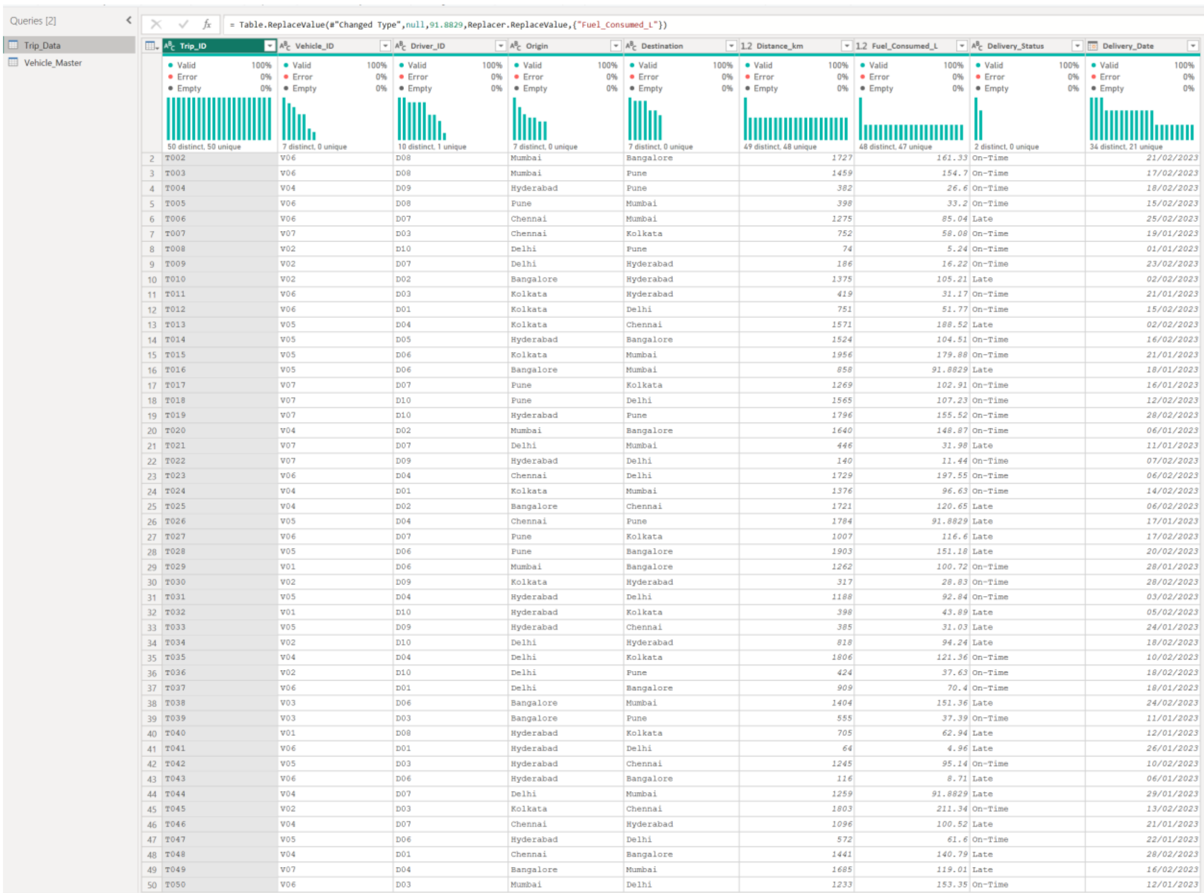
1. Open **Power BI Desktop → Home → Get Data → Excel workbook**.
2. Select your file → in the Navigator, check both **Trip_Data** and **Vehicle_Master** → click **Transform Data**

## STEP 2 — Clean Trip_Data in Power Query Editor

### 2a. Remove the blank rows

The sheet has ~950 fully empty rows after row 50. If you load these as-is, they'll appear as "blank" trips and wreck your counts/percentages.

1. In Power Query Editor, click on the **Trip_ID** column header dropdown.
2. Uncheck **(blank)** (or use **Home → Remove Rows → Remove Blank Rows**).
3. Confirm the row count at the bottom left now reads **50 rows**.

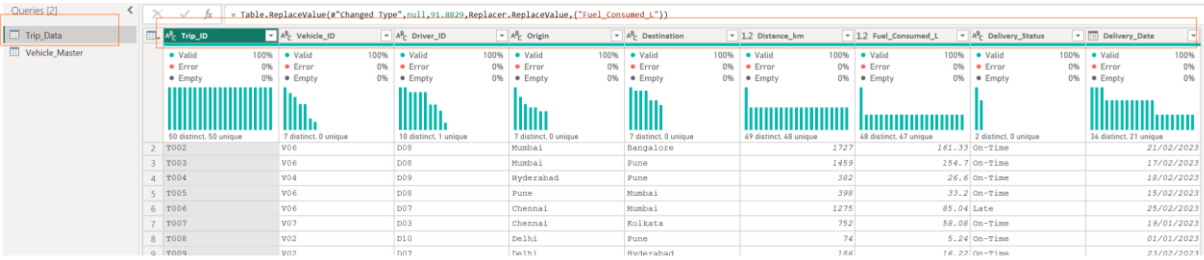



| | Trip_ID | Vehicle_ID | Driver_ID | Origin | Destination | Distance_km | Fuel_Consumed_L | Delivery_Status | Delivery_Date |
|---|---|---|---|---|---|---|---|---|---|
| | Valid 100% Error 0% Empty 0% | Valid 100% Error 0% Empty 0% | Valid 100% Error 0% Empty 0% | Valid 100% Error 0% Empty 0% | Valid 100% Error 0% Empty 0% | Valid 100% Error 0% Empty 0% | Valid 100% Error 0% Empty 0% | Valid 100% Error 0% Empty 0% | Valid 100% Error 0% Empty 0% |
| | 50 distinct, 50 unique | 7 distinct, 0 unique | 10 distinct, 1 unique | 7 distinct, 0 unique | 7 distinct, 0 unique | 49 distinct, 48 unique | 48 distinct, 47 unique | 2 distinct, 0 unique | 34 distinct, 21 unique |
| 2 | T002 | V06 | D08 | Mumbai | Bangalore | 1727 | 161.33 | On-Time | 21/02/2023 |
| 3 | T003 | V06 | D08 | Mumbai | Pune | 1459 | 154.7 | On-Time | 17/02/2023 |
| 4 | T004 | V04 | D09 | Hyderabad | Pune | 382 | 26.6 | On-Time | 18/02/2023 |
| 5 | T005 | V06 | D08 | Pune | Mumbai | 398 | 33.2 | On-Time | 15/02/2023 |
| 6 | T006 | V06 | D07 | Chennai | Mumbai | 1275 | 85.04 | Late | 25/02/2023 |
| 7 | T007 | V07 | D03 | Chennai | Kolkata | 752 | 58.08 | On-Time | 19/01/2023 |
| 8 | T008 | V02 | D10 | Delhi | Pune | 74 | 5.24 | On-Time | 01/01/2023 |
| 9 | T009 | V02 | D07 | Delhi | Hyderabad | 186 | 16.22 | On-Time | 23/02/2023 |
| 10 | T010 | V02 | D02 | Bangalore | Hyderabad | 1375 | 105.21 | Late | 02/02/2023 |
| 11 | T011 | V06 | D03 | Kolkata | Hyderabad | 419 | 31.17 | On-Time | 21/01/2023 |
| 12 | T012 | V06 | D01 | Kolkata | Delhi | 751 | 51.77 | On-Time | 15/02/2023 |
| 13 | T013 | V05 | D04 | Kolkata | Chennai | 1571 | 188.52 | Late | 02/02/2023 |
| 14 | T014 | V05 | D05 | Hyderabad | Bangalore | 1524 | 104.51 | On-Time | 16/02/2023 |
| 15 | T015 | V05 | D06 | Kolkata | Mumbai | 1956 | 179.88 | On-Time | 21/01/2023 |
| 16 | T016 | V05 | D06 | Bangalore | Mumbai | 858 | 91.8829 | Late | 18/01/2023 |
| 17 | T017 | V07 | D07 | Pune | Kolkata | 1269 | 102.91 | On-Time | 16/01/2023 |
| 18 | T018 | V07 | D10 | Pune | Delhi | 1565 | 107.23 | On-Time | 12/02/2023 |
| 19 | T019 | V07 | D10 | Hyderabad | Pune | 1796 | 155.52 | On-Time | 28/02/2023 |
| 20 | T020 | V04 | D02 | Mumbai | Bangalore | 1640 | 148.87 | On-Time | 06/01/2023 |
| 21 | T021 | V07 | D07 | Delhi | Mumbai | 446 | 31.98 | Late | 11/01/2023 |
| 22 | T022 | V07 | D09 | Hyderabad | Delhi | 140 | 11.44 | On-Time | 07/02/2023 |
| 23 | T023 | V06 | D04 | Chennai | Delhi | 1729 | 197.55 | On-Time | 06/02/2023 |
| 24 | T024 | V04 | D01 | Kolkata | Mumbai | 1376 | 96.63 | On-Time | 14/02/2023 |
| 25 | T025 | V04 | D02 | Bangalore | Chennai | 1721 | 120.65 | Late | 06/02/2023 |
| 26 | T026 | V05 | D04 | Chennai | Pune | 1784 | 91.8829 | Late | 17/01/2023 |
| 27 | T027 | V06 | D07 | Pune | Kolkata | 1007 | 116.6 | Late | 17/02/2023 |
| 28 | T028 | V05 | D06 | Pune | Bangalore | 1903 | 151.18 | Late | 20/02/2023 |
| 29 | T029 | V01 | D06 | Mumbai | Bangalore | 1262 | 100.72 | On-Time | 28/01/2023 |
| 30 | T030 | V02 | D09 | Kolkata | Hyderabad | 317 | 28.83 | On-Time | 28/02/2023 |
| 31 | T031 | V05 | D04 | Hyderabad | Delhi | 1188 | 92.84 | On-Time | 03/02/2023 |
| 32 | T032 | V01 | D10 | Hyderabad | Kolkata | 398 | 43.89 | Late | 05/02/2023 |
| 33 | T033 | V05 | D09 | Hyderabad | Chennai | 385 | 31.03 | Late | 24/01/2023 |
| 34 | T034 | V02 | D10 | Delhi | Hyderabad | 818 | 94.24 | Late | 18/02/2023 |
| 35 | T035 | V04 | D04 | Delhi | Kolkata | 1806 | 121.36 | On-Time | 10/02/2023 |
| 36 | T036 | V02 | D10 | Delhi | Pune | 424 | 37.63 | On-Time | 18/02/2023 |
| 37 | T037 | V06 | D01 | Delhi | Bangalore | 909 | 70.4 | On-Time | 18/01/2023 |
| 38 | T038 | V03 | D06 | Bangalore | Mumbai | 1404 | 151.36 | Late | 24/02/2023 |
| 39 | T039 | V03 | D03 | Bangalore | Pune | 555 | 37.39 | On-Time | 11/01/2023 |
| 40 | T040 | V01 | D08 | Hyderabad | Kolkata | 705 | 62.94 | Late | 12/01/2023 |
| 41 | T041 | V06 | D01 | Hyderabad | Delhi | 64 | 4.96 | Late | 26/01/2023 |
| 42 | T042 | V05 | D03 | Hyderabad | Chennai | 1245 | 95.14 | On-Time | 10/02/2023 |
| 43 | T043 | V06 | D06 | Hyderabad | Bangalore | 116 | 8.71 | Late | 06/01/2023 |
| 44 | T044 | V04 | D07 | Delhi | Mumbai | 1259 | 91.8829 | Late | 29/01/2023 |
| 45 | T045 | V02 | D03 | Kolkata | Chennai | 1803 | 211.34 | On-Time | 13/02/2023 |
| 46 | T046 | V04 | D07 | Chennai | Hyderabad | 1096 | 100.52 | Late | 21/01/2023 |
| 47 | T047 | V05 | D06 | Hyderabad | Delhi | 572 | 61.6 | On-Time | 22/01/2023 |
| 48 | T048 | V04 | D01 | Chennai | Bangalore | 1441 | 140.79 | Late | 28/02/2023 |
| 49 | T049 | V07 | D04 | Bangalore | Mumbai | 1685 | 119.01 | Late | 16/02/2023 |
| 50 | T050 | V06 | D03 | Mumbai | Delhi | 1233 | 153.35 | On-Time | 12/01/2023 |

## 2b. Fix data types

Select each column → **Transform** → **Data Type**:

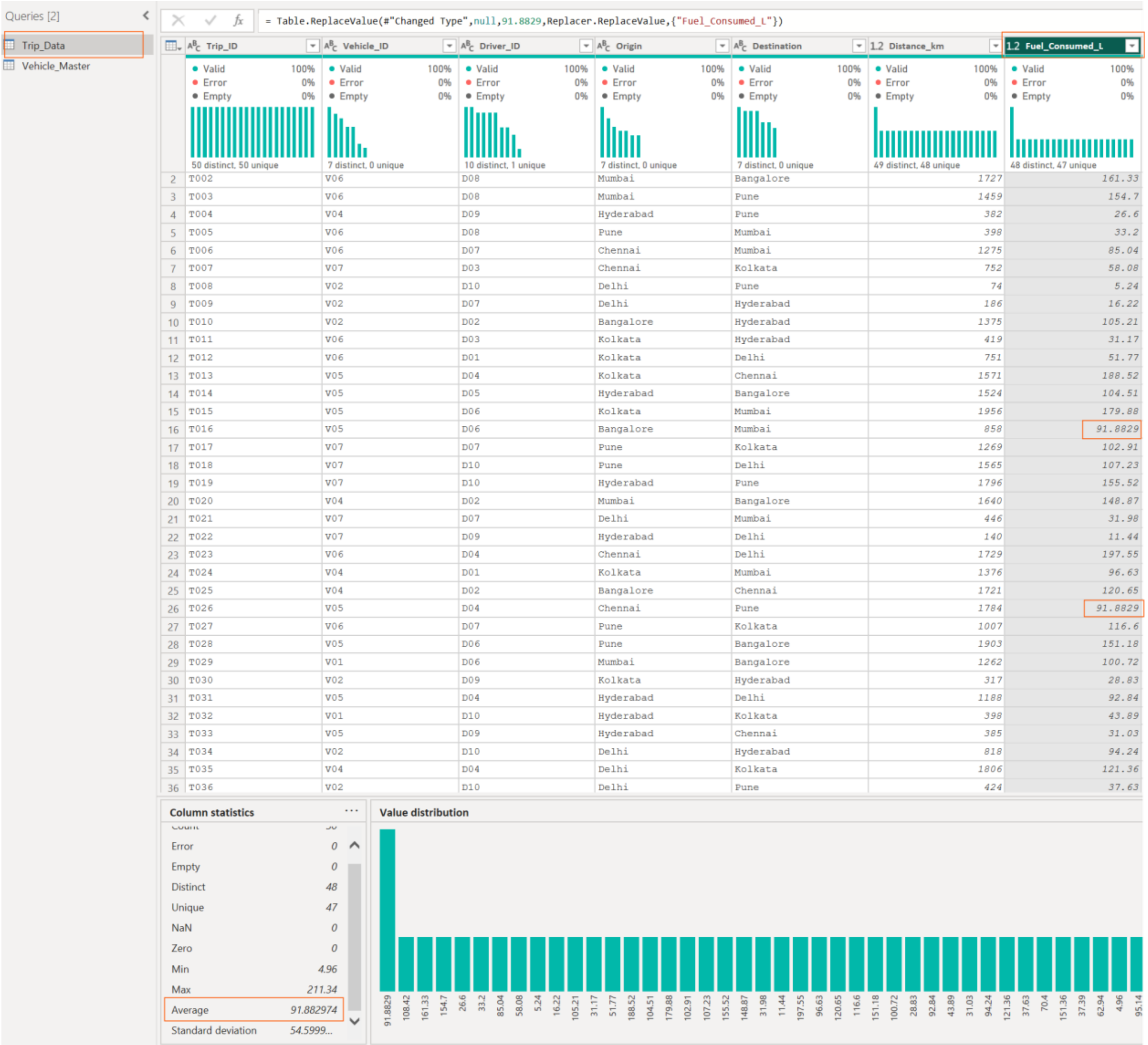
- Distance_km, Fuel_Consumed_L → Decimal Number
- Delivery_Date → Date
- Trip_ID, Vehicle_ID, Driver_ID, Origin, Destination, Delivery_Status → Text




## 2c. Mean imputation for missing Fuel_Consumed_L

Only 3 rows are missing this value. Power Query's "Fill Down" fills with the *previous* row, which is NOT mean imputation — so use **Replace Values** with the calculated mean instead:

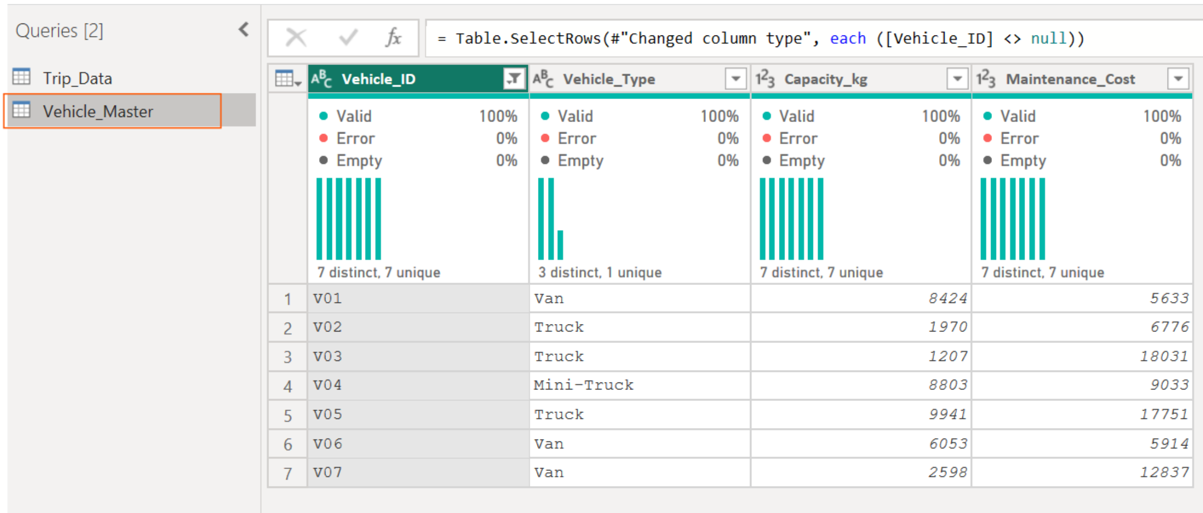
1. First find the mean: right-click the Fuel_Consumed_L column → **View** → **Column Profile**, or temporarily add a step and check **Transform** → **Statistics** → **Average** on that column (do this on the 47 non-blank values). Note the number down.
2. Go back to the applied step *before* you computed the average.
3. Select the Fuel_Consumed_L column → **Transform** → **Replace Values**:
    - Value to Find: leave blank (or use **Replace Errors**/filter for null)
    - Replace With: the mean value you noted




## 2d. Vehicle_Master

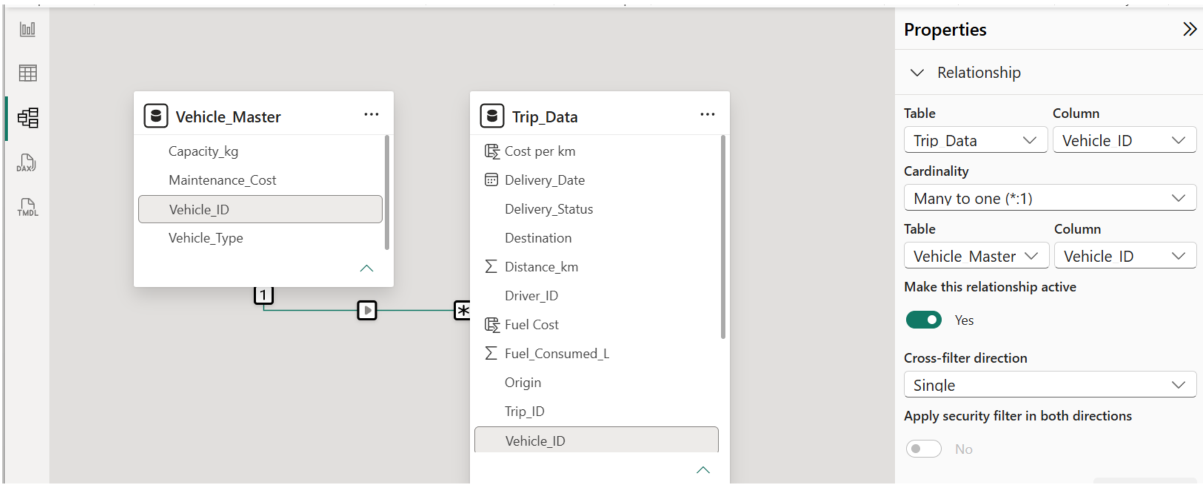
No cleaning needed — 7 rows, no missing values. Just confirm data types: Vehicle_ID/Vehicle_Type → Text, Capacity_kg/Maintenance_Cost → Decimal/Whole Number.

Click **Close & Apply**.



## STEP 3 — Build the Relationship

1. Go to the **Model** view (left sidebar icon).
2. Drag Vehicle_ID from **Trip_Data** onto Vehicle_ID in **Vehicle_Master** (or **Home → Manage Relationships → New**).
3. Confirm: Cardinality = **Many-to-One** (many trips per vehicle), Cross filter direction = **Single** (Vehicle_Master filters Trip_Data).



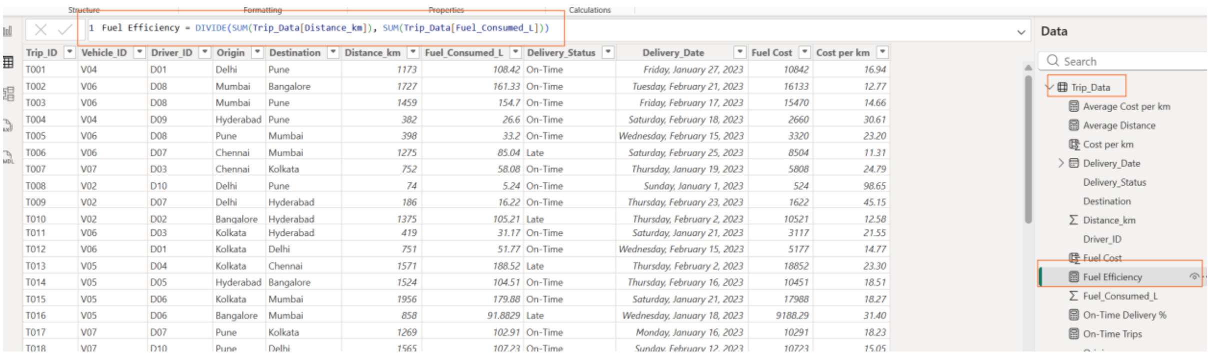

## STEP 4 — DAX Measures & Columns

Go to **Table view** or use the **Modeling → New Column / New Measure** ribbon buttons.

### 4a. Fuel Efficiency (measure — used in the line chart trend)

DAX

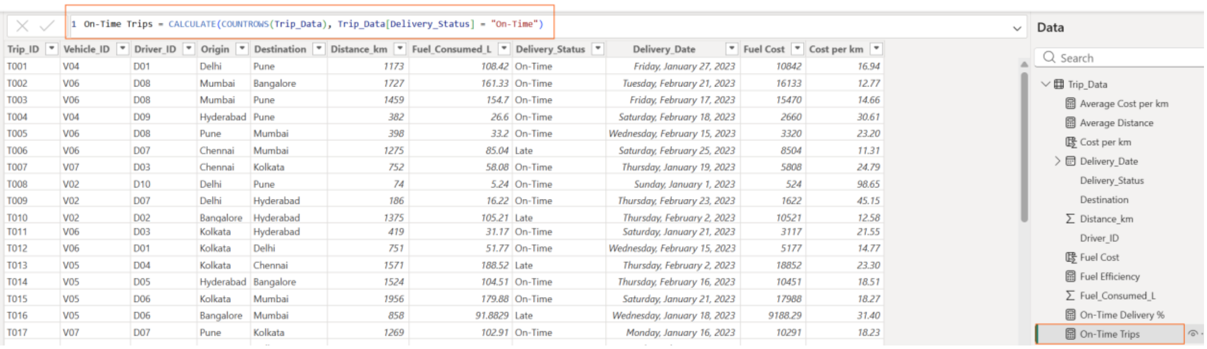
Fuel Efficiency = DIVIDE(SUM(Trip_Data[Distance_km]), SUM(Trip_Data[Fuel_Consumed_L]))



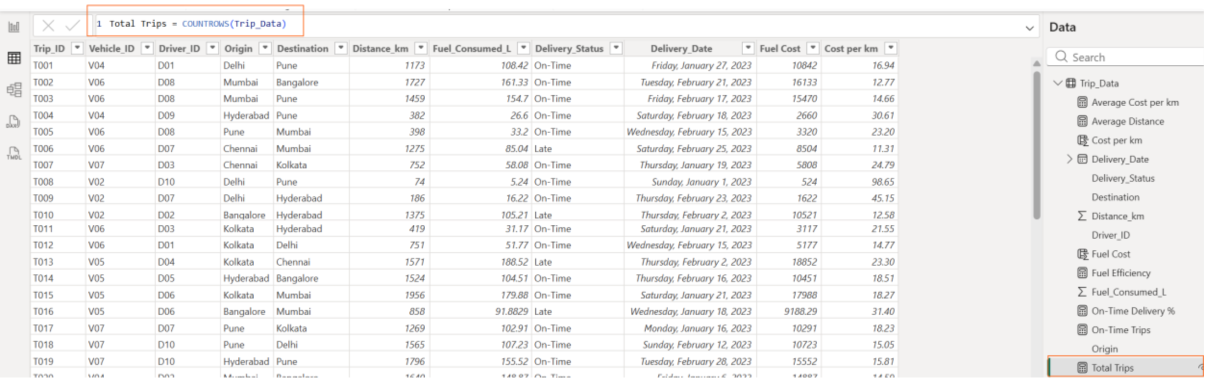

### 4b. On-Time Delivery % (measure)

DAX

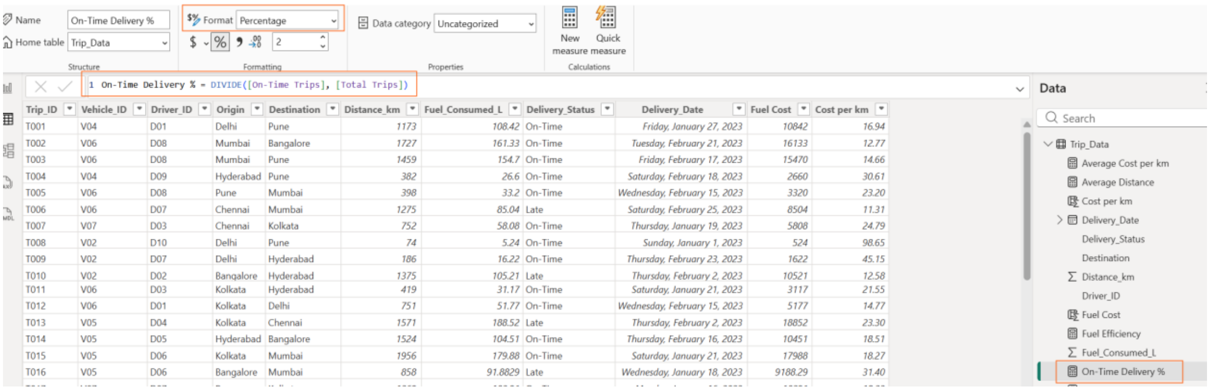
On-Time Trips = CALCULATE(COUNTROWS(Trip_Data), Trip_Data[Delivery_Status] = "On-Time")




Total Trips = COUNTROWS(Trip_Data)




On-Time Delivery % = DIVIDE([On-Time Trips], [Total Trips])




Format On-Time Delivery % as a percentage (Modeling tab → Format → Percentage).

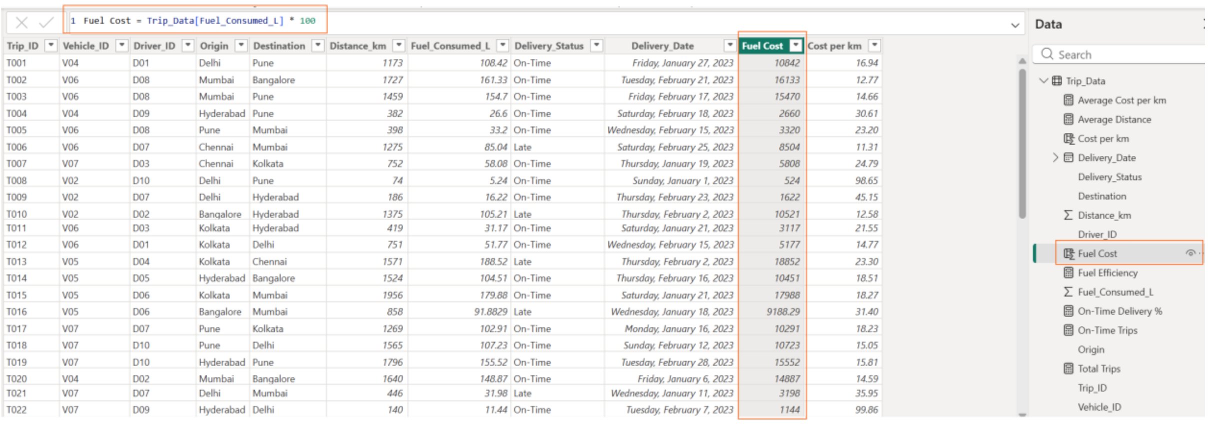
### 4c. Cost per km

This needs a calculated **column** first (so it's row-level, per trip — this is what the Decomposition Tree will explain), then a measure for the card/pie chart.

Fuel cost is noted as **100** (per liter):

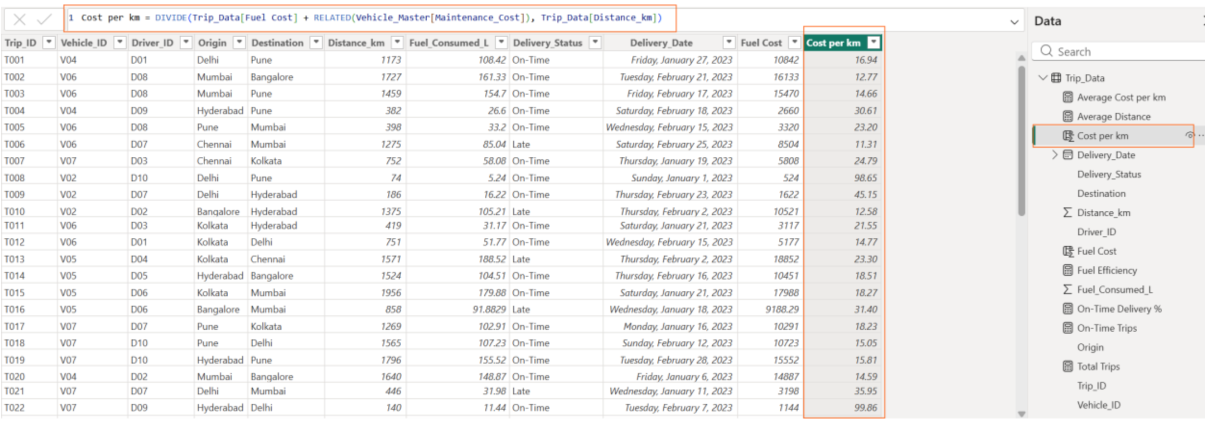
DAX

Fuel Cost = Trip_Data[Fuel_Consumed_L] * 100




| Trip_ID | Vehicle_ID | Driver_ID | Origin | Destination | Distance_km | Fuel_Consumed_L | Delivery_Status | Delivery_Date | Fuel Cost | Cost per km |
|---|---|---|---|---|---|---|---|---|---|---|
| T001 | V04 | D01 | Delhi | Pune | 1173 | 108.42 | On-Time | Friday, January 27, 2023 | 10842 | 16.94 |
| T002 | V06 | D08 | Mumbai | Bangalore | 1727 | 161.33 | On-Time | Tuesday, February 21, 2023 | 16133 | 12.77 |
| T003 | V06 | D08 | Mumbai | Pune | 1459 | 154.7 | On-Time | Friday, February 17, 2023 | 15470 | 14.66 |
| T004 | V04 | D09 | Hyderabad | Pune | 382 | 26.6 | On-Time | Saturday, February 18, 2023 | 2660 | 30.61 |
| T005 | V06 | D08 | Pune | Mumbai | 398 | 33.2 | On-Time | Wednesday, February 15, 2023 | 3320 | 23.20 |
| T006 | V06 | D07 | Chennai | Mumbai | 1275 | 85.04 | Late | Saturday, February 25, 2023 | 8504 | 11.31 |
| T007 | V07 | D03 | Chennai | Kolkata | 752 | 58.08 | On-Time | Thursday, January 19, 2023 | 5808 | 24.79 |
| T008 | V02 | D10 | Delhi | Pune | 74 | 5.24 | On-Time | Sunday, January 1, 2023 | 524 | 98.65 |
| T009 | V02 | D07 | Delhi | Hyderabad | 186 | 16.22 | On-Time | Thursday, February 23, 2023 | 1622 | 45.15 |
| T010 | V02 | D02 | Bangalore | Hyderabad | 1375 | 105.21 | Late | Thursday, February 2, 2023 | 10521 | 12.58 |
| T011 | V06 | D03 | Kolkata | Hyderabad | 419 | 31.17 | On-Time | Saturday, January 21, 2023 | 3117 | 21.55 |
| T012 | V06 | D01 | Kolkata | Delhi | 751 | 51.77 | On-Time | Wednesday, February 15, 2023 | 5177 | 14.77 |
| T013 | V05 | D04 | Kolkata | Chennai | 1571 | 188.52 | Late | Thursday, February 2, 2023 | 18852 | 23.30 |
| T014 | V05 | D05 | Hyderabad | Bangalore | 1524 | 104.51 | On-Time | Thursday, February 16, 2023 | 10451 | 18.51 |
| T015 | V05 | D06 | Kolkata | Mumbai | 1956 | 179.88 | On-Time | Saturday, January 21, 2023 | 17988 | 18.27 |
| T016 | V05 | D06 | Bangalore | Mumbai | 858 | 91.8829 | Late | Wednesday, January 18, 2023 | 9188.29 | 31.40 |
| T017 | V07 | D07 | Pune | Kolkata | 1269 | 102.91 | On-Time | Monday, January 16, 2023 | 10291 | 18.23 |
| T018 | V07 | D10 | Pune | Delhi | 1565 | 107.23 | On-Time | Sunday, February 12, 2023 | 10723 | 15.05 |
| T019 | V07 | D10 | Hyderabad | Pune | 1796 | 155.52 | On-Time | Tuesday, February 28, 2023 | 15552 | 15.81 |
| T020 | V04 | D02 | Mumbai | Bangalore | 1640 | 148.87 | On-Time | Friday, January 6, 2023 | 14887 | 14.59 |
| T021 | V07 | D07 | Delhi | Mumbai | 446 | 31.98 | Late | Wednesday, January 11, 2023 | 3198 | 35.95 |
| T022 | V07 | D09 | Hyderabad | Delhi | 140 | 11.44 | On-Time | Tuesday, February 7, 2023 | 1144 | 99.86 |

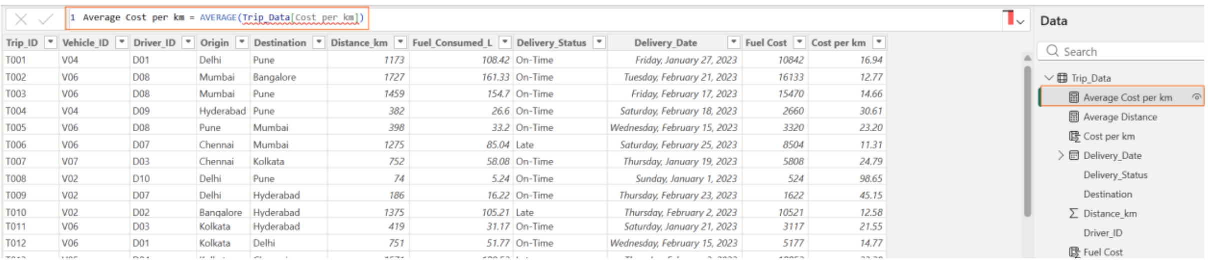
Cost per km = DIVIDE(Trip_Data[Fuel Cost] + RELATED(Vehicle_Master[Maintenance_Cost]), Trip_Data[Distance_km])




Then the card measure:

DAX

Average Cost per km = AVERAGE(Trip_Data[Cost per km])



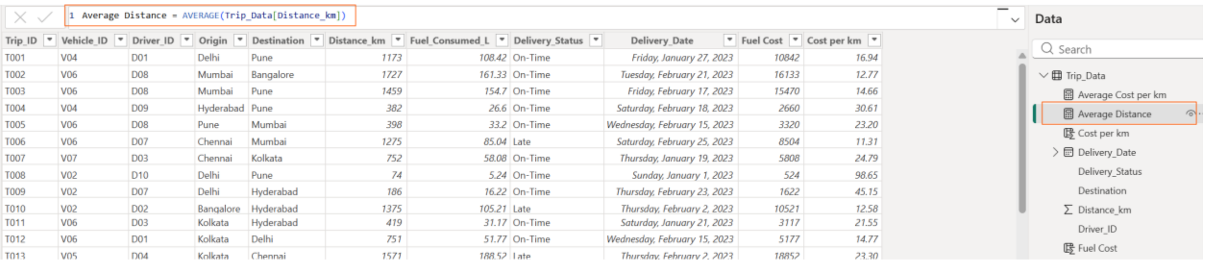

### 4d. Average Delivery Time — data limitation

The dataset only has **one date per trip** (Delivery_Date) — there's no pickup/start date or duration field, so a true "delivery time" can't be derived. Two options:

- **Recommended substitute:** replace this card with **Average Distance (km)**:

DAX

Average Distance = AVERAGE(Trip_Data[Distance_km])



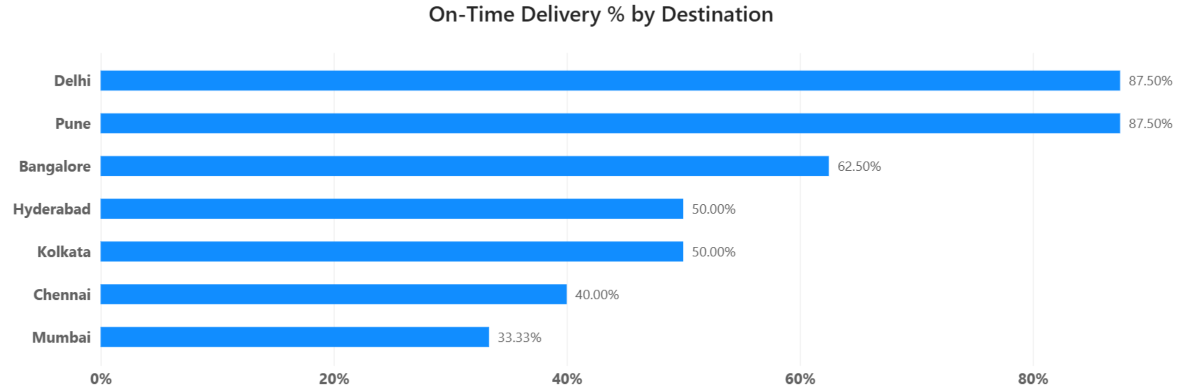

| Trip_ID | Vehicle_ID | Driver_ID | Origin | Destination | Distance_km | Fuel_Consumed_L | Delivery_Status | Delivery_Date | Fuel Cost | Cost per km |
|---|---|---|---|---|---|---|---|---|---|---|
| T001 | V04 | D01 | Delhi | Pune | 1173 | 108.42 | On-Time | Friday, January 27, 2023 | 10842 | 16.94 |
| T002 | V06 | D08 | Mumbai | Bangalore | 1727 | 161.33 | On-Time | Tuesday, February 21, 2023 | 16133 | 12.77 |
| T003 | V06 | D08 | Mumbai | Pune | 1459 | 154.7 | On-Time | Friday, February 17, 2023 | 15470 | 14.66 |
| T004 | V04 | D09 | Hyderabad | Pune | 382 | 26.6 | On-Time | Saturday, February 18, 2023 | 2660 | 30.61 |
| T005 | V06 | D08 | Pune | Mumbai | 398 | 33.2 | On-Time | Wednesday, February 15, 2023 | 3320 | 23.20 |
| T006 | V06 | D07 | Chennai | Mumbai | 1275 | 85.04 | Late | Saturday, February 25, 2023 | 8504 | 11.31 |
| T007 | V07 | D03 | Chennai | Kolkata | 752 | 58.08 | On-Time | Thursday, January 19, 2023 | 5808 | 24.79 |
| T008 | V02 | D10 | Delhi | Pune | 74 | 5.24 | On-Time | Sunday, January 1, 2023 | 524 | 98.65 |
| T009 | V02 | D07 | Delhi | Hyderabad | 186 | 16.22 | On-Time | Thursday, February 23, 2023 | 1622 | 45.15 |
| T010 | V02 | D02 | Bangalore | Hyderabad | 1375 | 105.21 | Late | Thursday, February 2, 2023 | 10521 | 12.58 |
| T011 | V06 | D03 | Kolkata | Hyderabad | 419 | 31.17 | On-Time | Saturday, January 21, 2023 | 3117 | 21.55 |
| T012 | V06 | D01 | Kolkata | Delhi | 751 | 51.77 | On-Time | Wednesday, February 15, 2023 | 5177 | 14.77 |
| T013 | V05 | D04 | Kolkata | Chennai | 1571 | 188.52 | Late | Thursday, February 2, 2023 | 18852 | 23.30 |

## STEP 5 — Visualizations

Switch to **Report view**.

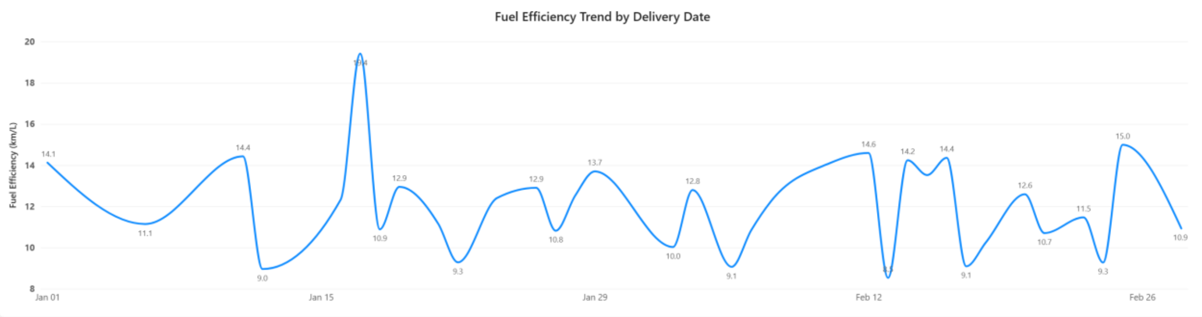
### 1. Bar chart — On-Time Delivery % by Destination

- Visual: **Clustered Bar Chart**
- Axis: Trip_Data[Destination]
- Values: [On-Time Delivery %]



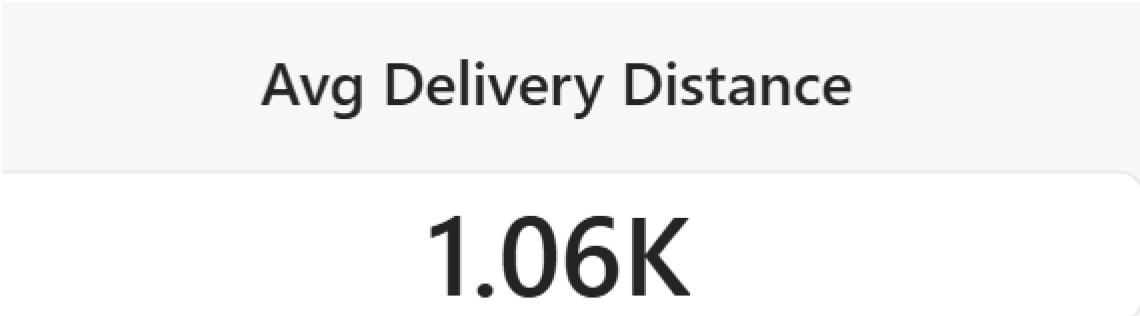

### 2. Line chart — Fuel efficiency trend by delivery date

- Visual: **Line Chart**
- X-axis: Trip_Data[Delivery_Date] (set to continuous date, not date hierarchy, for a true trend line)
- Y-axis: [Fuel Efficiency]



### 3. Cards

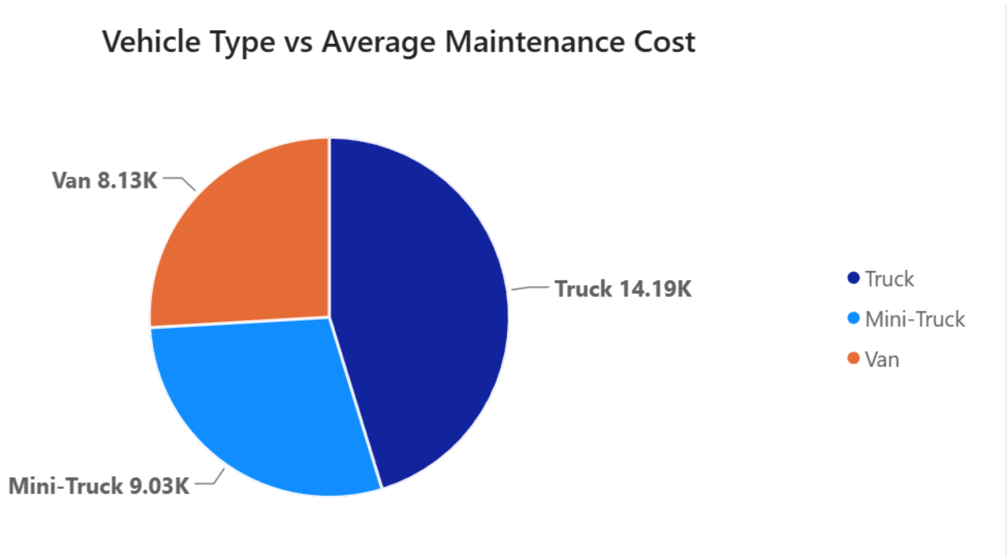
- **Card 1: [Average Distance]**




- **Card 2: [Average Cost per km]**


### 4. Pie chart — Vehicle type vs Average maintenance cost

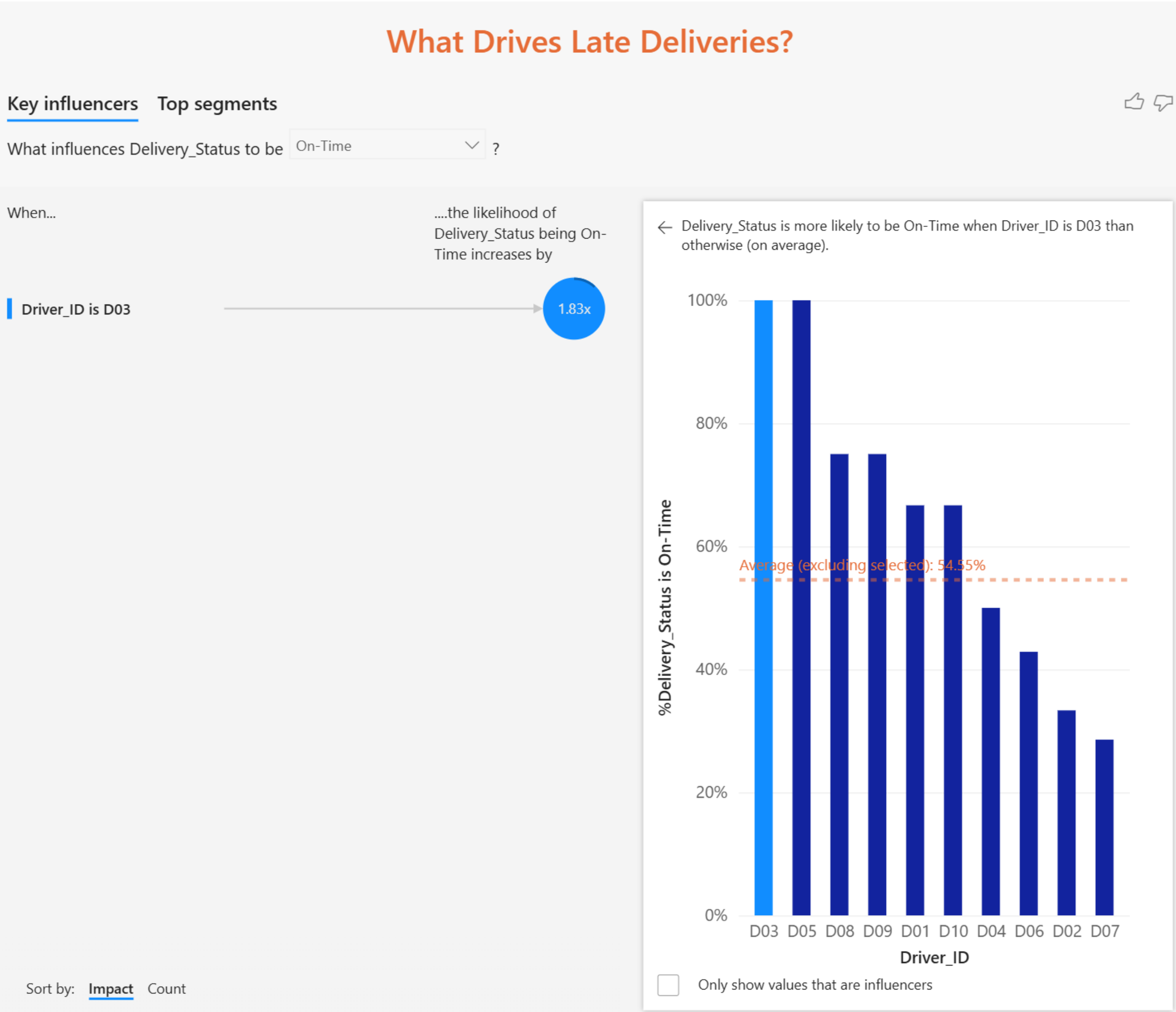
- Visual: **Pie Chart**
- Legend: Vehicle_Master[Vehicle_Type]
- Values: Vehicle_Master[Maintenance_Cost] → set aggregation to **Average**



## STEP 6 — AI-Powered Visuals

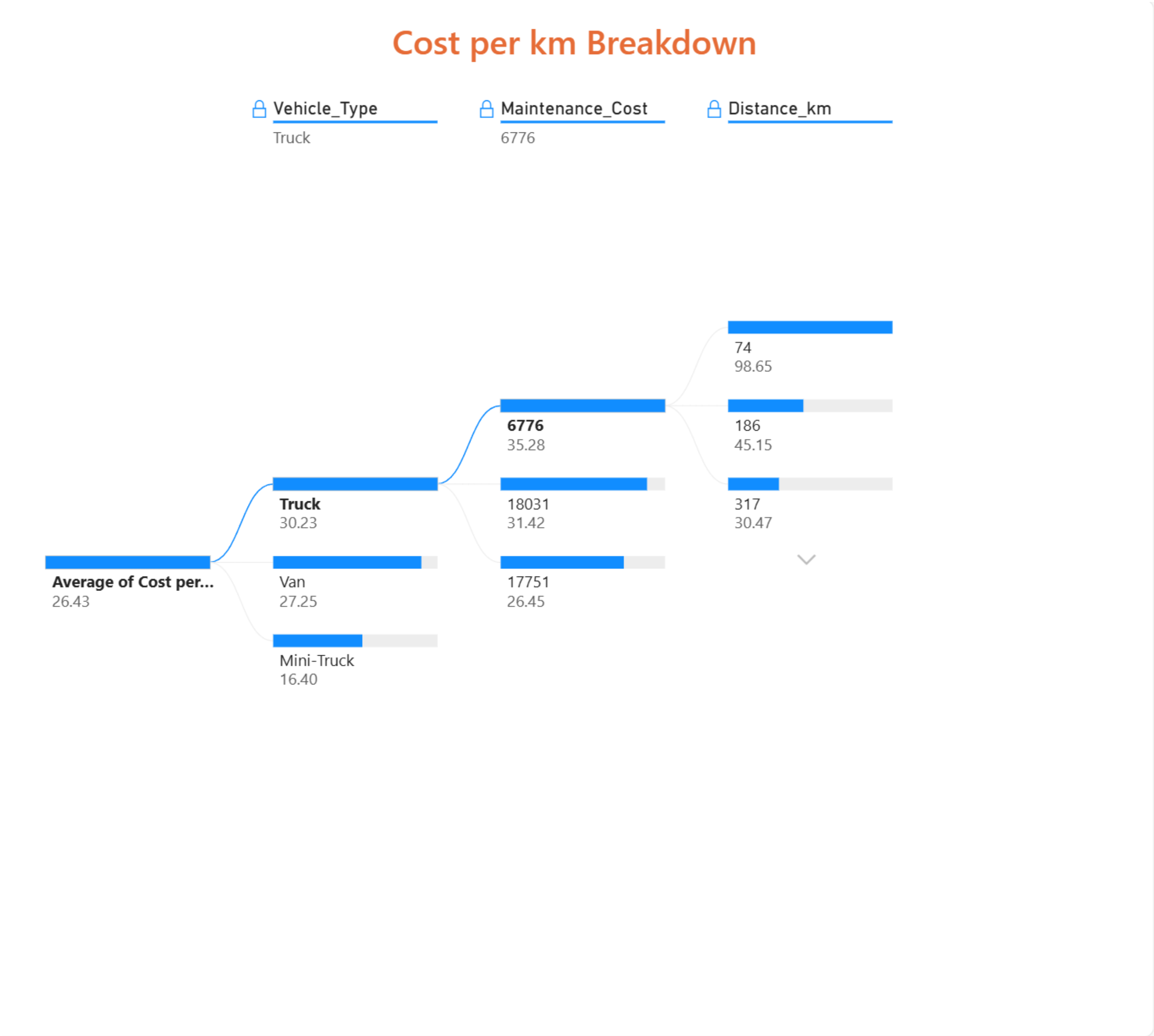
### 6a. Key Influencers Visual

1. Drag the **Key Influencers** icon onto the canvas.
2. **Analyze**: Trip_Data[Delivery_Status]
3. **Explain by**: add Distance_km, Vehicle_Type (drag from Vehicle_Master), and Driver_ID.
4. Power BI will show which factors most influence a trip being "Late" vs "On-Time" — click through the influencer bars to see the breakdown (e.g. "Distance > 1000 km increases likelihood of Late by X times").




### 6b. Decomposition Tree

1. Drag the **Decomposition Tree** icon onto the canvas.
2. **Analyze (Values well)**: Trip_Data[Cost per km] — set aggregation to Average.
3. **Explain by**: add Vehicle_Type, Maintenance_Cost, Distance_km (in that order, or let the tree's AI-split "High value" / "Low value" buttons pick the best splits automatically).
4. Click the + on a node and try the **AI splits (High/Low value)** icon to auto-find which factor most changes Cost per km.




### STEP 7 — Slicers

- Add slicers for Vehicle_Type and Delivery_Status so users can filter the whole page.